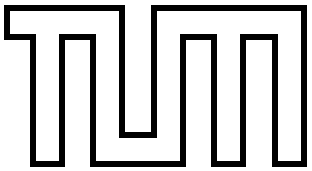


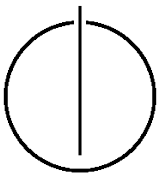
FAKULTÄT FÜR INFORMATIK

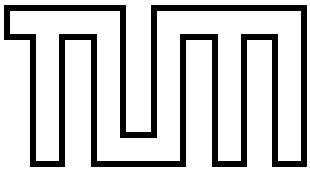
DER TECHNISCHEN UNIVERSITÄT MÜNCHEN

Bachelorarbeit in Informatik

Numerical Stability and Scalability of Secure Private Linear Programming

Raphael Arias





FAKULTÄT FÜR INFORMATIK

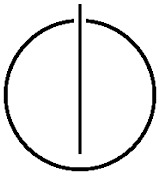
DER TECHNISCHEN UNIVERSITÄT MÜNCHEN

Bachelorarbeit in Informatik

Numerical Stability and Scalability of Secure Private Linear Programming

Numerische Stabilität und Skalierbarkeit von Verfahren zur sicheren und vertraulichen Linearen Programmierung

Author:	Raphael Arias
Supervisor:	Prof. Dr. Claudia Eckert
Advisor:	Krista Grothoff, M. Sc.
Date:	December 16, 2013



I assure the single handed composition of this bachelor's thesis only supported by declared resources.

München, den December 16, 2013 Raphael Arias

Acknowledgments

First of all, I would like to thank my supervisor Prof. Dr. Claudia Eckert for giving me the opportunity to work on this challenging and interesting topic as a bachelor's thesis, and being available to me when I needed support.

I also want to thank Florian Kerschbaum, one of the authors of the original paper that introduced the particular transformation-based algorithm for secure linear programming on which this work focuses. He was available to explain some details of their algorithm that I could not quite grasp at that time. His answers were very helpful to set me on the right track, pointing me towards some very good background reading and clarifying some unclear points.

Additionally, I would like to thank Christian Grothoff and Prof. Dennis Bennett who gave me some very useful advice and ideas.

Most importantly, I want to thank my advisor Krista Grothoff for the incredible support she gave me while I was working on this thesis. The weekly (and often more frequent) meetings were always helpful to discuss ideas and get feedback for my work. She was always willing to take some time and understand the problems that I was having with some details of related work or a particular part of the implementation. I can say this with a fair amount of certainty: she was the best advisor anyone can hope for.

Abstract

Linear programming (LP) has numerous applications in different fields. In some scenarios, e.g. supply chain master planning (SCMP), the goal is solving linear programs involving multiple parties reluctant to sharing their private information. In this case, methods from the area of secure multi-party computation (SMC) can be used. Secure multi-party versions of LP solvers have been known to be impractical due to high communication complexity. To overcome this, solutions based on problem transformation have been put forward.

In this thesis, one such algorithm, proposed by Dreier and Kerschbaum, is discussed, implemented, and evaluated with respect to numerical stability and scalability. Results obtained with different parameter sets and different test cases are presented and some problems are exposed. It was found that the algorithm has some unforeseen limitations, particularly when implemented within the bounds of normal primitive data types. Random numbers generated during the protocol have to be extremely small so as to not cause problems with overflows after a series of multiplications. The number of peers participating additionally limits the size of numbers. A positive finding was that results produced when none of the aforementioned problems occur are generally quite accurate. We discuss a few possibilities to overcome some of the problems with an implementation using arbitrary precision numbers.

Contents

Outline of the Thesis

Part I: Introduction and Theory

CHAPTER 1: INTRODUCTION

In this chapter, an introduction and motivation of the topic are given. Relevant related work is discussed.

CHAPTER 2: BACKGROUND

The background for important concepts that are necessary for the understanding of the problem is presented.

Part II: Evaluating Secure Private Linear Programming

CHAPTER 3: SECURE PRIVATE LINEAR PROGRAMMING

This chapter presents the algorithm by Dreier and Kerschbaum and gives some details about the implementation.

CHAPTER 4: TEST DESIGN

In this chapter, the test design is explained in detail. Some target points in the protocol are defined.

CHAPTER 5: TEST RESULTS

This chapter presents results of the tests, as well as a discussion and a conclusion.

Part I.

Introduction and Theory

1. Introduction

Linear Programming (LP) has many applications in diverse fields, such as production planning, network flow, game theory, among others. One of the applications is supply chain optimization and, more specifically, supply chain master planning (SCMP), which is used by a group of collaborating companies wishing to globally optimize their supply chain process. The motivation for this is quite simple: By collaborating with other companies to efficiently use all resources available, all participants can often benefit, saving resources, money, or time. A practical example of this, given by Vaidya [29], is that of a number of businesses that started saving money by merging loads of different companies with the same destination using an online collaborative logistics service; trucks that were travelling empty before (after having discharged their cargo at the destination) were used to transport different goods to the next destination, effectively eliminating (or decreasing the number of) detours. In that case, the load information was outsourced to a third party (the online service).

The difficulty with this is that the companies were often reluctant to share some of their private data, such as costs or capacity constraints, with the other companies. Two main alternatives for dealing with this problem have been researched. One is to solve the linear program directly using a secure multi-party variant of the Simplex method (or another LP solving algorithm). The other is based on problem transformation, meaning the actual linear program the companies have is transformed in such a way that makes it impossible or at least difficult to extract any company's private data.

The motivation for the transformation-based approach comes from the fact that, to this point, secure collaborative linear programming has been impractical for performance reasons, as discussed later.

In this work, one transformation-based algorithm will be examined and evaluated with respect to numerical stability and scalability properties. This particular algorithm was chosen due to its status as one of the most recent and correct transformation-based approaches, and because it has addressed and corrected some of the flaws of previous methods, making it a promising candidate for practical applications. The authors affirmed that stability and scalability testing was work remaining to be done, and this thesis aims to achieve that goal. It should be remarked, as a side note, that such analysis of additional approaches would have far exceeded the scope of a bachelor's thesis.

In the remainder of the first part of this thesis, section 1.1 will give an overview of related work, while chapter 2 will present some background on the different components of this topic. In the second part, chapter 3 will give a detailed description to the specific algorithm examined in this work, as well as some important notes regarding the implementation, and finally a comprehensive description of test design. In chapter 5, the outcomes of these tests will be presented, and a summary of the implications of these findings will be discussed.

1.1. Related Work

As mentioned above, in the field of secure linear programming, there are two distinct general approaches. One approach is the collaborative computation of an optimal solution using a secure version of the simplex method. Li and Atallah [19] have proposed such a protocol for two parties, which is a quite limited scenario.

Toft [28] has proposed a secure multi-party version of the Simplex algorithm which has proved to be impractical for real applications. Dreier and Kerschbaum estimate their prototypical implementation of Toft's protocol would take 7 years to solve their use case LP problem [10].

A more efficient approach has been proposed by Catrina and De Hoogh [7] using fixed-point arithmetic.

The transformation-based approach to secure linear programming was used by Du [11] in a secure two-party environment; however, he provided no security analysis and his protocol only applied to the two-party scenario. Vaidya [29] proposed a solution based on Du's transformation. In his scenario, one party holds the objective function, while the other party knows the constraints. This is usually not a realistic scenario, and Bednarz et al. [2] found that Vaidya's transformation admits infeasible solutions to the LP under some circumstances. Dreier and Kerschbaum [10] propose a transformation based on Vaidya's work, which does not make any limiting assumptions about the partitioning of objective function and constraints. They provide a formal security analysis in terms of leakage quantification. Their transformation is the main focus of this work. Laud and Pankova [17, 18] present attacks on the different transformation-based approaches.

Numerical stability testing is not always an easy task and approaches have usually been rather problem-specific. In the past few years, however, research has been conducted to find generic numerical stability testing solutions. Rowan [22] developed a method for stability testing which compares algorithm results obtained using double precision with results obtained using single precision. Knizia et al. [16] describe a method for stability testing that consists of comparing an algorithm's output with the output produced by the same program when injecting code randomizing rounding modes of floating point operations into the compiler output. Higham [13] proposed a method to find pathological input data. He uses a direct search method to find inputs that optimize an objective function to maximize numerical error. The input for which numerical error is maximal is obviously a pathological input for the algorithm. The objective function Higham optimizes is usually problem-specific and needs to be determined first. Kahan et al. [14] run a program with different rounding modes and measure the differences in the results. On a well-conditioned problem, large differences would indicate a numerically unstable algorithm. Zhou [32] combines Higham's and Kahan's approach: Kahan's method is used to find an objective function that can then be maximized using Higham's method.

2. Background

In this section we will introduce some important concepts that are necessary to understand the transformation presented in chapter 3 and the evaluation of it. In section 2.1, some basics of linear programming are introduced. We will introduce and motivate supply chain master planning in section 2.2. In section 2.3, we discuss some fundamental concepts of secure multi-party computation. In section 2.4, important techniques for securely sharing secrets are discussed. We will especially examine how computations on shared secrets can be performed, as this has a major impact on the implementation (and limitations) of the algorithm under consideration.

2.1. Linear programming

In Linear Programming (LP), a problem consists of a linear objective function, a set of variables x and a set of linear constraints for these variables. There can be equality constraints as well as inequality constraints. Depending on the problem, the goal is to either maximize (in the context of profit) or minimize (in the context of costs) the objective function. Usually, all variables should be non-negative. A linear programming (minimization) problem can thus be represented as

$$\begin{aligned} & \min c^T x & \\ \text{s.t.} \quad & M_1 x & = b_1 \\ & M_2 x & \leq b_2 \\ & x & \geq 0 \end{aligned}$$

where c is the cost vector of size k, x is a variable vector of size k, M_1 is a $m_1 \times k$ matrix representing the equality constraints in conjunction with b_1, an m_1-vector, and M_2 (an $m_2 \times k$ matrix) and b_2 (an m_2-vector) represent the inequality constraints.

To solve linear programs, the Simplex method by George B. Dantzig has long been the algorithm of choice [8, 9]. Even though it has an exponential worst case complexity, it performs efficiently in most practical cases.

The Simplex method finds an optimal solution to a linear program by traversing the vertices of the n-polytope forming the feasible area described by the constraints. By moving to the next vertex with the smallest value, the algorithm steadily approaches the optimal solution [21]. This approach is correct, as it can be shown that if the linear program has an optimal solution value, it (also) takes this value on a vertex of the polytope. The method solves linear programs that are in *standard form* (no inequality constraints, all variables greater or equal to 0). However, transforming a non-standard LP to standard form, is not difficult [21] (slack variables can be used, as will be discussed in section 3.1).

2.2. Supply chain master planning

Supply chain master planning refers to the "coordination of activities and processes along a supply chain" [27] and is a sub-field of supply chain management. The main characteristic of supply chain master planning is that it refers to a process of planning and coordination on the *entire* supply chain. Optimizing activities on the entire supply chain can be greatly beneficial, as it might serve to speed up processes, save capacities, make more efficient use of resources, better meet customer demands, and thus, reduce costs and/or maximize gains.

This sort of coordination obviously requires information about the entire supply chain. This can be problematic if the supply chain does not completely belong to one large company. When the supply chain is shared between different companies, it is rare that a single one of those businesses owns all necessary information about the supply chain to perform master planning. The reason for this is that businesses are often reluctant to share their private information, as they fear that it might be used to their disadvantage at a later point. Production costs or available capacities are information that if disclosed can have a negative influence on a company's negotiation position [7].

In supply chain optimization, linear programming is very important, as many problems in this field can be modelled as linear programs. Resources, ingredients, and products can be modelled as variables, while customer demand as well as transport and storage capacities can be expressed using linear (equality/inequality) constraints. Analogously, the costs (for production, acquisition, transport, etc.) can be modelled by the cost function or vector.

In master planning, the linear program is not owned by a single company but distributed among the individual participants. This means that each business will own some variables and constraints, some of which might even be shared between multiple companies. For instance, company A might produce some resource that company B needs. Then the variable that corresponds to that product is shared between companies A and B. Company A might have constraints on the variable (cannot produce more than X_1), whereas company B has other constraints (need at least X_2 per production cycle). Together, these (and many other) constraints form a global linear problem. In this work, a way to solve such a linear program privately, without e.g. disclosing the values of X_1 and X_2 to other parties, will be examined.

2.3. Secure multi-party computation

Secure multi-party computation (SMC) refers to the collaborative computation of a function without revealing private inputs to the other parties. As an example, consider a group of employees at a company who want to know which of them has the largest salary. They do not want to reveal how much they receive and still learn who earns the most. Using secure multi-party computation, the group can compute the function ($\max$) without disclosing their private input (salary).

More precisely, the involved parties (sometimes also referred to as players or peers) will not learn more about the secret inputs than what can be deduced from the output of the computed function in conjunction with their own secret input [20]. This definition

is necessary, as depending on the function it might be possible to deduce the input from the output of the function. For an extreme example, consider the case where two parties jointly compute the XOR-function $\oplus : \{0,1\}^2 \to \{0,1\}$ on their private inputs. Let b_A and b_B be the secret input of players A and B respectively. Then the result $r = b_A \oplus b_B$ will enable player A to calculate $b_B = b_A \oplus r$ and thus learn the other player's secret input.

In the remainder of this section, we will briefly introduce some historical background.

2.3.1. History

The field of secure multi-party computation was introduced by Yao [31] with the example of his famous millionaires' problem, where two millionaires would like to know which of them is richer, without telling the other their actual account balance. Similar concepts had been discussed before by Shamir et al. [26] when introducing protocols to play "mental poker", and by Blum who introduced "coin flipping via telephone" [5].

The techniques developed in this field have been viewed as theoretical rather than practical for a long time, the difficulty being that solutions to these problems require a extremely large amount of communication between involved parties. In the more recent past, as technology has advanced and devices possess more and more computing power (and communications bandwidth has increased), the protocols are becoming suitable for practical applications. This has led to an increase in research in this area.

2.3.2. Protocols

There have long been generalized protocols to compute any function in a secure multi-party way. This can be done with garbled circuits introduced by Yao [31]. The idea is that any function can be expressed as a circuit and this circuit can be transformed ("garbled") using a one-way function and then inputs can be encoded and evaluated in the garbled circuit.

The existence of so many specific protocols for particular problems is due to the fact that the general protocol usually involves a much higher communication complexity that can often be improved given some specialized knowledge of the problem domain.

Many of the secure multi-party protocols rely on a technique called secret sharing. This technique will be examined in the next section.

2.4. Secret sharing

In this section, the idea of secret sharing will be introduced. This is a crucial concept that plays an extremely important role in the algorithm presented in chapter 3.

Secret sharing is a commonly-used approach in secure multi-party computations. The term "secret sharing" might be misleading in the way that it suggests a secret is shared with and thus disclosed to another party; in fact, secret sharing is not the simple act of telling somebody one's secret, but a much more powerful tool.

Consider the following scenario: Alice, Bob, and Carol are scientists who discover a very important fact. They consider it to be too dangerous to humanity, so they decide not to openly disclose it to the world. They also want to make sure that none of them alone

can show the secret to anybody else. They decide on *splitting* the secret into various *shares* in a way such that only the three of them together are able to recover it, should they ever feel that humanity is now ready to learn the truth. Then they destroy the original secret.

There are different ways to share a secret. The focus here lies on a type known as threshold schemes. Consider a different scenario in which Alice wants to split a personal secret into different parts and hand some shares (sometimes these are also referred to as shadows) to Bob and Carol. She does not want to be dependent on both of them to recover the secret at a later time. She wants it to suffice if either Bob or Carol is available when she recovers the secret. Alice can use a 2-out-of-3-threshold scheme ((2,3)-threshold scheme) and create three shares, hand one to Bob and one to Carol and recover the secret either together with Bob or with Carol. However, now also Bob and Carol can recover Alice's secret without Alice, which is probably not what she intended. To prevent this, she can use a (3,4)-threshold scheme and keep two shares for herself, while handing one each to Bob and Carol.

In the following, different threshold secret sharing schemes will be described.

2.4.1. Polynomial interpolation scheme (Shamir)

The polynomial interpolation scheme was first introduced by Shamir [25] and is widely known as *Shamir secret sharing*. In this secret sharing scheme, the secret is encoded as constant part of a polynomial. In a (k,n)-threshold scheme, the polynomial has degree $k-1$ and n shares are created by evaluating the polynomial at n distinct x-coordinates. The idea is that to interpolate a polynomial of degree $k-1$ we need k points on the curve. With only $k-1$ points no information about the constant part of the polynomial, and thus the secret, is revealed.

To share a secret S using this scheme, a polynomial $p(x)$ of degree $k-1$ is picked randomly

$$p(x) = a_0 + a_1 x + \cdots + a_{k-1} x^{k-1}$$

with $a_0 = S$. To generate the n shares of the secret, $p(x)$ is evaluated at n distinct positions. To recover the secret, at least k of these shares are needed. The polynomial can then be interpolated and evaluated at $p(0) = a_0 = S$.

This scheme can be used in any field $\mathbb{F}$ and there are a few optimized implementations that work in GF(256). While these are practical for applications such as splitting an RSA private key into several shares, they also limit the scheme in some ways. A few of these limitations will be discussed in more detail in section 3.2.1. One that should be mentioned here is the fact, that the field size directly limits the number of shares and thus the number of parties that may participate. Formally for the number n of shares, it holds that $n < |\mathbb{F}|$. This is clear, as for every player a distinct position x_i is needed to evaluate the polynomial $p(x)$ at. In a field of size $|\mathbb{F}|$ there are only as many distinct elements. However, 0 clearly cannot be used to create shares, as $p(0) = S$, so $n = |\mathbb{F}|$ cannot hold.

2.4.2. Karnin-Greene-Hellman

This secret sharing scheme uses matrix multiplication to create a (m,n)-threshold scheme. It was proposed by Karnin et al. [15]. For a given secret S they choose $n+1$ different

m-dimensional vectors $a_i, i \in \{0, \ldots, n\}$, such that any subset of m vectors form a $m \times m$-matrix of full rank [24]. Then a random $m+1$-dimensional (row-)vector u is chosen from the set of vectors which satisfy $S = u \cdot a_0$. Subsequently the n shadows $v_i = u \cdot a_i, i \in \{1, \ldots, n\}$ are distributed and a_0 is made public. Any m shadows can be used to construct a system of linear equations that can be solved to reveal u. $S = u \cdot a_0$ can then be computed, since a_0 is public.

2.4.3. Computations on shared secrets

To be consistent with other literature, in this part a t-out-of-n-threshold scheme will refer to a scheme where a group consisting of up to t parties is **not** able to recover the secret from their shares. This means that for Shamir secret sharing a secret is encoded into a polynomial of degree t (so $t+1$ shares are needed for recovery).

Some secret sharing schemes have a very interesting and useful property: they allow computations on the shares of the secrets. Formally, if secrets s_A and s_B are distributed in n shares each over the parties $P_i, i \in \{1, \ldots, n\}$ as $p_{A,i}$ and $p_{B,i}$ respectively, each party P_i can locally compute $p_{A,i} + p_{B,i}$ (without communication with the other parties) resulting in a new secret $s_A + s_B$ which can be recovered when $t+1$ parties (assuming a (t,n)-threshold scheme) combine their shares. Similarly, secrets can be scaled (multiplied with a scalar value) and multiplied (by another secret). In the multiplication case, some special attention is necessary, since it cannot be computed individually as it requires communication and the computations that have to be performed are rather complex.

We consider the computation on Shamir shared values, as proposed by Ben-Or et al. [3]. Since Shamir shared values are points of a polynomial and assuming each party P_i receives an evaluation of the polynomial at point x_i two secrets A and B encoded by the polynomials $f(x)$ and $g(x)$ can be added. Every party P_i computes $h(x_i) = f(x_i) + g(x_i)$. The parties now hold the shares of the polynomial $h(x)$. Since $h(0) = f(0) + g(0)$ the secret encoded by $h(x)$ is the value of the sum of the secrets A and B as intended.

Scaling secrets (multiplication with a scalar) is similarly trivial. Assuming the parties want to compute γA, where A is the secret shared between the parties and every party P_i owns a share $f(x_i)$ of the polynomial $f(x)$ that encodes A. Each party P_i just computes $\gamma f(x_i)$ (again, locally, without communication with other participants). The resulting polynomial that is shared between the parties is $\gamma f(x)$ and thus since $f(0) = A$, immediately follows $\gamma f(0) = \gamma A$.

However, multiplication of several secrets creates a problem. When multiplying two polynomials of degree t (which is what we have in a (t,n)-threshold scheme), the result is a polynomial of degree $2t$. To interpolate this polynomial we would need $2t+1$ shares, but we might only have t, if $n = t$. Even if we choose $n = 2t+1$, a series of consecutive multiplications will force the degree of the polynomial out of bounds. To prevent this, a degree reduction step is performed by Ben-Or et al. [3] after multiplications on shares.

2.4.4. Multiplication and degree reduction

For this part the following conditions are necessary: We assume a (t, n)-threshold scheme where $t = \left\lfloor \frac{n-1}{2} \right\rfloor$. Let A and B be two secrets represented by $f(x)$ and $g(x)$, respectively. Each of the n parties P_i owns one share of each of the two polynomials, corresponding to

the evaluation of the polynomial at this party's point. That is, party P_i knows the values $f(x_i)$ and $g(x_i)$. Let $f(x) \cdot g(x) = h(x)$. Because $deg(f(x)) = deg(g(x)) = t$, $deg(h(x)) = 2t$, we are able to obtain n points of the polynomial $h(x)$ of degree $2t$ if each party multiplies its shares $f(x_i) \cdot g(x_i)$. At this point, since $t = \left\lfloor \frac{n-1}{2} \right\rfloor$, the degree of the polynomial is $2t \leq 2(\frac{n-1}{2}) = n - 1$, and thus at most n shares are needed (and available) to reconstruct the secret. Since after one multiplication the result is reconstructible without further computations, the explanations given below are often omitted. For this work, however, they are absolutely necessary, as we will see.

So, what happens in the case that the secret should not be recovered at this point? There are cases (and the work done in this thesis is one of them) where the results of multiplications are intermediate results that should remain secret. Since the shares should also continue to be useful in further computations, the degree of the underlying polynomial has to be reduced back to t. The basic idea is to truncate the polynomial $h(x) = \sum_{j=0}^{2t} c_j x^j$ to $k(x) = \sum_{j=0}^{t} c_j x^j$, preserving $k(0) = h(0) = f(0) \cdot g(0)$.

A problem that persists is that neither $h(x)$ nor $k(x)$ are truly random polynomials over the field $\mathbb{F}$, as they are not irreducible (obviously, they are the product of two other polynomials). To re-randomize the polynomial, each party P_i generates shares of a random polynomial $q_i(x)$ of degree $2t$, with the property that $q_i(0) = 0$, that is, $q_i(x)$ encodes the secret 0. The parties distribute those (each party P_j receives the share $q_i(x_j)$) and individually locally add their shares of each q_i. The parties have now securely computed $q(x) = \sum_{i=1}^{n} q_i(x)$, where $q(0) = 0$ still holds and each party holds a share. The sum of the polynomials $h(x)$ and $q(x)$, a share of which each player now also computes locally with this share of $h(x)$ and the share of $q(x)$ he just computed, is now a random polynomial $h'(x)$ of degree $2t$ that encodes the same secret as $h(x)$.

This polynomial can then be truncated to obtain $k(x)$. The difficulty here is to do so without first reconstructing the polynomial $h'(x)$ (if $h'(x)$ were public, obtaining $k(x)$ would be trivial) as this intermediate result of the calculation should remain unknown.

According to Asharov and Lindell [1] it suffices to multiply the shares of $h'(x)$ (the literature refers to these as β_i – β-shares of which every player P_i owns one share β_i) with a constant matrix. Constant means that it is fixed for one run of the protocol since it is derived from the points (the x_i) which are known to all participants. The goal is to calculate the shares δ_i for each party P_i. The δ-shares are the shares of the polynomial $k(x)$ that the players want.

Let A be the aforementioned constant matrix, δ the n-vector of all the δ_i, and β the n-vector of all the β_i. Then

$$\delta = A\beta$$

or, more precisely, for every individual δ_i:

$$\delta_i = \sum_{j=1}^{n} A_{i,j}\beta_j$$

It is clear, that β is needed in its entirety to calculate a single share of δ for one party. It is also clear, that β cannot just be handed to party P_i to compute δ_i, as A is known to everybody and this would permit P_i to calculate *all* of δ. However, we are working with secret sharing and it has already been shown that addition and scaling of shares are not

difficult problems. This means that every player P_j can create shares of β_j and hand one each to all players. The player P_i can now scale each share of β he has received with the corresponding constant element of the matrix, $A_{i,j}$ and sum those values up to obtain his *share* of δ_i. He also does this for every other i. When everyone has a share of each δ_i, the shares are restored to their owner P_i who can then reconstruct his secret value, the δ_i. Note that this recovered secret value still corresponds to a share. It is player P_i's share of the polynomial $k(x)$, meaning it has the value $k(x_i)$.

The constant matrix A will now be examined in more detail. Asharov and Lindell [1] give A as follows:

$$A = V_\alpha \cdot P_T \cdot V_\alpha^{-1}$$

where V_α is the $n \times n$ Vandermonde matrix for the n-vector α constituted of all the points x_i, V_α^{-1} is its inverse, and $P_T \in \mathbb{F}^{n \times n}$ is the linear projection of the first t elements ($T = \{1, \dots t\}$) of an n-vector. For a proof of why this matrix has the desired properties, see [1].

2.5. Numerical Stability

Numerical stability usually refers to an algorithm's sensitivity to small errors in input data and how those affect the algorithm's result. This makes numerical stability similar to conditioning of a problem, but while condition is inherent in a general problem, stability is a property of a specific algorithm. If, in a well-conditioned problem, a slight perturbation of input values causes an extreme variation of the output, the algorithm is said to be numerically unstable.

Numerical stability problems often arise in floating point computations due to rounding errors and an effect called cancellation. Since in this scenario we examine an algorithm used for supply chain master planning, the linear programs we encounter are completely integral. This means that the common problems of floating point arithmetic are not immediately a problem. However, the solution to the transformed linear problem will almost never be integer. Details of this will be covered in section 3.2.1.

It needs to be pointed out that the approaches to numerical stability testing discussed in section 1.1 are not directly applicable in this scenario as they deal with numerical stability issues introduced by floating point computation and, as we will see further on, issues here arise from other problems. This is not entirely true for Higham's method. The approach to find pathological input data could be applied here, but it would be highly complex as the direct search method would have to tolerate different runs on random transformations.

2.5.1. Cancellation

Cancellation is the effect that occurs in floating point arithmetic when subtracting two numbers of similar magnitude and with the same sign. If the most significant digits get cancelled out, what remains as result is the difference of the values in the less significant digits. Assuming that the values being subtracted are results of earlier floating point computations it is likely that the less significant digits contain rounding errors. As the higher digits are cancelled out, the only significant digits in the result now are the ones that were subject to rounding error before. This means that the relative error has increased dramatically in this single subtraction. Cancellation can also occur when adding two numbers of

opposite signs with similar magnitude. Cancellation is generally best avoided by reducing the number of subtractions an algorithm needs. This can often be done by rearranging the operations.

An example of this, taken from [6], is the problem of finding the larger root of the quadratic equation $x^2 + 2px - q = 0$, given $p = 500, \quad q = 1$ when computing with 5 (decimal) digits of accuracy. The formula $x := \sqrt{p^2 + q} - p$ yields $\sqrt{25001} - 500 = 0.00099999900\ldots$ when computing exactly. As only 5 digits are used, however, the result is 0. If the formula is transformed to $x := \frac{q}{\sqrt{p^2+q}+p}$, the result can be computed correctly (or as correctly as possible when only 5 digits of precision are available).

Part II.

Evaluating Secure Private Linear Programming

3. Secure Private Linear Programming

In this chapter the transformation-based approach to private linear programming proposed by Dreier and Kerschbaum [10], the main subject of this thesis, will be examined.

The chapter is organized as follows. In section 3.1, the problem transformation algorithm will be presented and a secure multi-party version will be introduced. In section 3.2, some implementation details will be discussed.

3.1. Problem Transformation

Dreier and Kerschbaum suppose a linear program in the form

$$\begin{aligned} \min & \, c^T x \\ \text{s.t. } M_1 x &= b_1 \\ M_2 x &\leq b_2 \\ x &\geq 0 \end{aligned}$$

where $c \in \mathbb{Z}^{n\times 1}, M_1 \in \mathbb{N}_0^{m_1\times n}, b_1 \in \mathbb{N}_0^{m_1\times 1}, M_2 \in \mathbb{N}_0^{m_2\times n}, b_2 \in \mathbb{N}_0^{m_2\times 1}$ and $x \in \mathbb{N}_0^{n\times 1}$. Note that integer and, more specifically, positive input problems (with exception of the cost function) are assumed. This is due to the fact that their specific use case is supply chain master planning, where assuming such inputs is realistic and practical. Non-integer inputs are thus not directly discussed here. Applying the protocol to such inputs should, however, be possible when using fixed-point arithmetic.

They then use a positive monomial matrix[1] $Q \in \mathbb{Z}^{n\times n}$ to hide c. The linear program then has the form

$$\begin{aligned} \min & \, c^T Q Q^{-1} x \\ \text{s.t. } M_1 Q Q^{-1} x &= b_1 \\ M_2 Q Q^{-1} x &\leq b_2 \\ Q^{-1} x &\geq 0. \end{aligned}$$

A positive vector $r \in \mathbb{N}_0^{n\times 1}$ is used to hide x. This produces a program as follows:

$$\begin{aligned} \min & \, c^T Q (Q^{-1} x + r) \\ \text{s.t. } M_1 Q (Q^{-1} x + r) &= b_1 + M_1 Q r \\ M_2 Q (Q^{-1} x + r) &\leq b_2 + M_2 Q r \\ (Q^{-1} x + r) &\geq r. \end{aligned}$$

[1] A matrix that has exactly one strictly positive element per row and column. All other elements are 0.

Substituting $z = Q^{-1}x + r$ (where $z \in \mathbb{Q}^{n\times 1}$) and using a strictly positive diagonal matrix[2] $S \in \mathbb{N}_0^{n\times n}$ they obtain

$$\begin{aligned} &\min c^T Q z \\ \text{s.t. } M_1 Q z &= b_1 + M_1 Q r \\ M_2 Q z &\leq b_2 + M_2 Q r \\ S z &\geq S r. \end{aligned}$$

Note that

$$Sz \geq Sr \qquad \Leftrightarrow \qquad -Sz \leq -Sr.$$

This is used to express all the inequality constraints in "$\leq$"-form.

They then rewrite the linear program as

$$\begin{aligned} &\min {c'_s}^T z_s \\ \text{s.t. } M' z_s &= b' \\ z_s &\geq 0 \end{aligned}$$

where ${c'_s}^T$ is $c^T Q$ with added zeros for the slack variables[3], $z_s \in \mathbb{Q}^{(n+m_2)\times 1}$ is z with added slack variables and

$$M' = \begin{pmatrix} M_1 Q & 0 \\ M_2 Q & \\ -S & \mathrm{A} \end{pmatrix}, \qquad b' = \begin{pmatrix} b_1 + M_1 Q r \\ b_2 + M_2 Q r \\ -Sr \end{pmatrix}$$

where $A \in \{0,1\}^{(m_2+n)\times(m_2+n)}$ is a permutation matrix[4] permuting the slack variables.

In the last step, they hide the contents of M' and b' using a non-singular random matrix $P \in \mathbb{Z}^{(m_1+m_2+n)\times(m1+m2+n)}$ to generate $M'' = P \cdot M', \quad b'' = P \cdot b'$. A solver then solves the linear program

$$\begin{aligned} &\min {c'_s}^T z_s \\ \text{s.t. } M'' z_s &= b'' \\ z_s &\geq 0. \end{aligned}$$

From the resulting optimal solution z_s it is possible to extract z (by taking the first n elements) and then calculate x since $x = Q(z - r)$.

[2] Diagonal matrix with all elements $a_{i,i}$ strictly positive.

[3] Slack variables are added to transform inequality constraints into equality constraints. E.g. $5x_1 \leq 23$ can be expressed as $5x_1 + s_1 = 23, s_1 \geq 0$.

[4] A monomial matrix with all non-zero elements equal to 1.

3.1.1. Secure multi-party computation of the transformation

Throughout this section, it is assumed that the transformation is performed by p parties. To make the transformation secure, all of these operations have to be computed in a secure setting. No party can know the entirety of M_1, M_2, b_1, b_2 and c since those are composed of each party's secret input. Furthermore, no party can know P, Q, S, A and r as that, in conjunction with the public M'', would allow the recovery of everyone's secret input. Instead, the elements of these matrices are shared amongst all partners and never owned by one party completely.

This is made possible by computations on shared secrets, as were previously discussed in section 2.4.3 and section 2.4.4. To compute P, each party i generates a random invertible matrix P_i and shares a part of each element of the matrix with all other parties. At the end of this process, each party has a share of each element of each of the P_i. By multiplying their shares (actually, doing matrix multiplication on the matrices composed of the shares of all elements) they compute $P = \prod_{i=0}^{p-1} P_i$ where every party now holds a share of each element of P. This works analogously for all other matrices and vectors; $Q = \prod_{i=0}^{p-1} Q_i$, $A = \prod_{i=0}^{p-1} A_i$, $S = \sum_{i=0}^{p-1} S_i$, $r = \sum_{i=0}^{p-1} r_i$. Note that S and r are computed using sums and not products, making S and r simpler and less expensive (in terms of both computation and communication costs) to compute.

In a similar way, the original linear program will also be shared between parties. Since every party has their own private equality and inequality constraints for the variables as well as cost function, M_1, b_1, M_2, b_2 and c are composed of individual constraints of the parties. The parties need to specify in advance which constraints belong to which party. Then each party can share the (elements of) matrices and vectors of their linear program. After that, M_1, b_1, M_2, b_2 and c are computed securely by adding everybody's shares. The results may not be reconstructed, since that would reveal constraints (as parties agreed upon which constraints belonged to which party or parties). Instead, now the transformation is performed and the resulting disguised linear program is given to an external solver. The external solver cannot tell the original problem, and its resultant solution for the vector z_s can then be used by the parties to securely compute $x = Q(z - r)$ (the slack variables can be removed to obtain z). The shares of the values of x can then be returned to each original owner, who then can reconstruct them to know his private result.

As will be discussed in more detail in section 3.2.1, the z_s obtained from the solver is not integral. To fit with the data that otherwise occurs here, it needs to be converted to an integer by scaling, leading to a sort of fixed-point arithmetic being performed.

In the next section, a limitation inherent to the multiplicative secret sharing used in the protocol is examined more closely.

Limitation on threshold

As was discussed in section 2.4.4, multiplicative secret sharing with the Shamir scheme requires that the threshold $t = \left\lfloor \frac{n-1}{2} \right\rfloor$, where n is the number of players. This means that when considering a straightforward implementation of the protocol, $t + 1$ players can reconstruct the secret, and thus a participating party in the protocol needs to trust $t + 1$ of the other parties to not collude and recover the secret data. Note that this is a inherent limitation of the multiplication. The first obvious solution — simply increasing the number

of shares created (remember what Alice did in section 2.4) — will not make this problem disappear. As this is not trivial, it will be explained in further detail here.

Assuming N parties (distinguishing the number of parties N now from the number of shares n), one might want to have the threshold $t = N - 1$, so that only all N players together can recover the secrets. As seen before, multiplication requires $t = \left\lfloor \frac{n-1}{2} \right\rfloor$ and thus $n \geq 2t + 1$. Substituting we obtain $n \geq 2(N-1) + 1 = 2N - 1$. Analogous to Alice's problem in section 2.4, if party P_i wants to always be included in the set of parties that reconstruct the secret, the party will hand out $N - 1$ shares to the other players and keep N shares for itself. Once the protocol reaches the point where computations have to be performed on the secret, the problem becomes clear.

To keep it simple, addition shall be used here as an example. Let A and B be the two secrets that are meant to be added. They are already distributed as shares among all N parties, every party P_i holding one share (of each secret) corresponding to the evaluation of the polynomials $f(x)$ and $g(x)$ at the point x_i. And (this is new) the two owners of the secrets (the parties that originally shared them – they shall be called P_A and P_B for obvious reasons) hold an additional $N - 1$ shares each. Two scenarios are possible: (1) The additional shares correspond to evaluations at different points – player P_A owns shares $f(x_j)$ for $x_j \in M_A$ while player P_B owns shares $g(x_j)$ for $x_j \in M_B$ where $M_A \neq M_B$. (2) The additional shares correspond to evaluations at the same points – $M_A = M_B$.

1) In this first scenario, nothing can be computed. The addition of secrets requires that all corresponding shares of the secrets be added. For evaluations at different points, it is unclear if any conclusions can be drawn. It is certain to be non-trivial to do so, however.

2) That leaves the second scenario, where the sets of points at which the polynomial is evaluated are equal. The challenge would then be to add the shares of player P_A and the shares of player P_B. Who should own the results? If player P_A learns them, he has N shares of the polynomial $h(x)$ ($N - 1$ from this calculation and 1 because everyone owns one share after adding their shares of f and g). As we remember, however, N shares are enough to reconstruct the secret. So no one individual of these players can learn the results of this computation. Though distributing them might seem to be an acceptable solution, distribution again leads to the problem that some subset of the parties can collude to learn the secret values, the problem we were trying to eliminate in the first place.

This example illustrates how the bound on t depending on n cannot be circumvented. While this is not a formal proof, it does illustrate the nature of problems that arise. A cleverer way to go about this might exist, but is unknown; further research in this direction would be interesting.

It is arguable whether this bound is acceptable for the application motivated in the algorithm we focus on. If a company is not inclined to disclose its data, it might also be reluctant to disclose shared secrets knowing that any $t + 1$ of their partners could collude to recover the data nevertheless.

3.2. Implementation

With the implementation of the protocol for this thesis, the focus was producing code that was easily testable. Throughout the whole process it was, however, also a goal to produce a useful code base that could be built upon, should the algorithm prove stable, scalable and useful.

The transformation algorithm was implemented in C. A library for secret sharing was developed in coordination with this implementation, although it is not part of this work. The new library was necessary, because existing secret sharing libraries (although theoretically supporting additive secret sharing) provided no way of multiplying secrets and did not allow any manipulation of field size, which was limited to 256 (GF(256)) in all observed cases. This was not acceptable, as it bounds the number of different secrets. The approach of splitting larger data into individual bytes to be shared (thus working in GF(256)) is also not practical, given that it destroys the additive and multiplicative properties of Shamir secret sharing.

3.2.1. Important notes

While secret sharing is relatively easy and addition and scaling of shared secrets can be performed by an external secret sharing library, for the multiplication of shared secrets the protocol needs to be a direct part of the program and cannot easily be outsourced to a library. This is due to the fact that the multiplication of shared secrets (as seen in section 2.4.4) requires communication between the parties and communication needs to be performed by the program, given that communication might be handled differently in different applications. The library used for this work tries to facilitate multiplicative secret sharing the most possible, which is a particularly challenging task[5].

As mentioned before, while the resulting vector x in the use case of supply chain master planning can be assumed to be completely integral, the vector z_s (or z) never can. This is due to its definition $z = Q^{-1}x + r$, where x and r are integer vectors and Q^{-1} is the inverse of a random positive monomial matrix, whose elements will only be integer if Q happens to be a permutation matrix. Which would make Q effectively useless. Thus, the elements of z have to be treated with special attention. Rounding them destroys valuable information the solver obtained and results in incorrect solutions when transformed back.

To prevent this, the elements of z are scaled up before they are shared between the parties. This is necessary, since the secret sharing library in use takes integer secrets. Scaling z with the factor γ in the equation above ($z = Q^{-1}x + r$), we obtain a slightly different result: $\gamma x = Q(\gamma z - \gamma r)$. Scaling the values of x back will give each party the appropriate results.

3.2.2. Number representation in the secret sharing library

The library works with all numbers within a finite prime field, the size of which can be specified when initializing the context. However, the field must be smaller than 2^{64} so

[5] Changes to such a library giving an API using callbacks for communications functions might resolve the problem of requiring the actual multiplication protocol to be implemented in each program, but at this time, no such library exists.

that its elements can be completely stored in a *long* variable[6]. Generally speaking, in our experiments, a field size of $2^{62} - 57$ was used. The upper half of the field size was reserved to represent negative values as follows: Let ${}_{64}\mathbb{Z}$ be the (signed) 64-bit integer number space and $\mathbb{F}$ a prime field of size p. Then the function $f : {}_{64}\mathbb{Z} \to \mathbb{F}$ describes how 64-bit integers are represented in the field.

$$f(z) = \begin{cases} z & \text{if } 0 \leq z \leq \frac{p-1}{2} \\ p + z & \text{otherwise.} \end{cases}$$

It is important to note that numbers that are to be represented in the field should be a reasonable fraction of the prime size, as to not overflow into the negative half of the field, or from the negative into the positive half. The prime size can be defined adequately, as to guarantee a large enough space. Values in the field which are slightly larger than $\frac{p-1}{2}$ are likely to be overflow values.However, as with many representations, it is the user's responsibility to ensure that overflow does not occur.

[6]This limitation was considered acceptable, as it was known before library development that the solvers used for this application only accepted `double` inputs and outputs.

4. Test Design

When testing an algorithm for numerical stability, we try to find cases for the input data that are prone to producing incorrect output. Ideally these input cases have a certain structure that can be determined. When working with matrices, it seems likely that bad conditioning of the input data should provoke errors in the output. If that can be confirmed, it is not necessarily a fault in the algorithm. An ill-conditioned problem is, of course, more sensitive than a well-conditioned one.

It is, of course, also possible that we find that the transformation produces ill-conditioned matrices for some specific, well-conditioned input. That would be a problem, especially if the input is a realistic input for some use case of the transformation, e.g. supply chain master planning. It might also be a problem if a large number of input cases having some specific structure would cause this.

When working with integers, numerical errors do not occur in rounding, as is common in a lot of algorithms using floating point arithmetic. In this case, closer attention has to be paid to overflow and underflow problems. Test cases should reflect this problem by including numbers that are likely to over- or underflow and thus result in incorrect output.

Furthermore, constructing near singularity in integer matrices is not as straightforward as with floating point numbers. There are, however, some possibilities to approximate near singularity[1].

One approach is to generate a matrix with two identical columns and change one of the columns adding or subtracting 1 to one element.

Another possibility is to search for a matrix with a determinant close to (but different from) 0 using a random walk approach minimizing the determinant. This can be achieved by cycling through an initially random matrix and randomly adding or subtracting 1 to an element.

In addition to problems that can occur with integers, such as under- and overflow, the result of the transformation which is fed to an external solver can contain numbers that cause complications to that solver. LP solvers generally use floating point numbers even if the input and the solution are integral. If numbers generated during the transformation are too large to be precisely represented as floating point numbers, or if during the execution of the simplex method they become too large, the solution to the transformed problem will not be correct.

Another problem that can occur is that the LP solver might have to perform operations on the numbers, that are numerically problematic. These can be divisions of very large numbers by very small numbers or subtractions of two numbers. The question is whether an input problem can have properties such that the output of the transformation provokes this behaviour in the solver.

[1] These were suggested by Dennis Bennett [4].

4.1. Target points

In order to test the behaviour of the transformation we will target the following points in the protocol.

a) The input data - the linear program to be transformed. Does a slight perturbation of input data, e.g. column swapping, provoke large differences in the re-transformed solution of the transformed problem?

b) The random matrices created during the protocol. Are there "unlucky" cases that transform a well-conditioned problem into an ill-conditioned problem? Can they cause overflows if they contain large random numbers?

c) Number of peers. A higher number of peers will result in a higher number of matrix multiplications, thus producing larger numbers that might cause overflows, both during and after the transformation (in integer as well as floating point number spaces).

d) Random numbers in general. How large are they allowed to be before overflows occur? Smaller random numbers might avoid overflows, at the cost of reducing security of the transformation[2].

e) The LP solver. Can the transformation generate cases where the LP solver has to perform numerically problematic operations, such as subtractions or division by small numbers[3].

In the following section we describe how test cases that stress these points in the protocol will be generated.

4.2. Test case generation

In this section the test case generation process is described. The different approaches to generating adequate testing data will be discussed.

The approach for targeting the aforementioned points in the protocol:

a) 1) Generate a linear program for which the optimal solution is known. Does the transformation yield a correct result?

2) Does the solution vary for the same test case being transformed with different random matrices?

3) Permute columns/lines and try again. Is the result the same (only permuted?)

4) Generate a near-singular linear program as described before. Solve it directly using an LP solver and compare the result delivered by the protocol.

[2] Actually, the security according to Dreier and Kerschbaum's analysis decreases with increasing number of factors [10], making reasonably small values not necessarily bad. Very small values, however, might be problematic.

[3] This was proposed in conversations with Christian Grothoff [12].

b) Instead of randomly generating matrices during the protocol, generate pathological cases. How likely is it that this happens by accident?[4]

c) Test with a different number of peers.

d) Reiterate the tests with different sizes of random numbers and compare the results.

e) No tests specifically directed at this point will be generated. However, influences of this could be observable in the other tests.

To generate and run the tests proposed here, a test script was produced, written in Python 3.2. The script generates random linear programs with a known solution. This is achieved by generating M_1, M_2, x and c at random and then calculating $b_1 = M_1 x$, $b_2' = M_2 x$, and optionally $b_2 = b_2' + \gamma$ for some vector γ (to transform the constraints into actual inequality constraints). This way the linear program is guaranteed to have a feasible solution: x. The solution might not be optimal, so an optimal solution for the LP is then calculated by an external solver[5].

The test program then runs the transformation program on the generated test case, and the external solver provides a solution for the transformed problem. This solution is fed back to the transformation program which in turn computes the final x and outputs it. The test program calls the transformation repeatedly on the same input case to evaluate how much the random matrices used influence the output. This corresponds to point a.2) in the above enumeration. The test case, the solution, and the output of the transformation program (transformed LP, transformed-back solution) are saved.

After all runs on the same test case, the script analyses the obtained results and computes some metrics which will be discussed in more detail in section 5.1.3.

To fully evaluate the transformation protocol, some parameters in the transformation program also have to be adjusted and tests have to repeated with these different parameters. These parameters specifically are c) the number of peers and d) size of random numbers. To ensure that the differences in measurements do not depend on the input cases, the test cases generated before can be reloaded and run again.

To achieve testing a.3) and a.4) of the above enumeration, the script provides options to swap columns and lines in the test cases, and to generate a near-singular linear program.

[4] Testing this target point explicitly turned out to be unnecessary; normal random matrices sufficed to indicate problematic situations, as is explained later.

[5] The solver SoPlex (`http://soplex.zib.de/soplex.shtml`), which was originally developed in the context of [30], and which is distributed under the ZIB Academic License (`http://scip.zib.de/academic.txt`), was used.

5. Test Results

In this chapter, the results obtained from the testing process will be presented and discussed. In section 5.3, we will draw some conclusions from the results, indicate what future work could be interesting and describe some problems that were encountered during this work.

5.1. Results obtained

In this section, the results of the testing process are presented. We make the distinction between findings that concern the correctness of the algorithm and findings that are related to scalability. It will become clear that scalability issues have a large impact on the outcome of the protocol and thus greatly influence the correctness.

5.1.1. Correctness and stability concerns

The tests that were executed largely show that a correct, although sometimes slightly perturbed, solution can be recovered from the protocol. The problems that occurred here are mostly related to problems of scalability which will be discussed in the next section.

With respect to the target points discussed in the previous chapter, this is what has been found:

a) The input data - the linear program to be transformed. This turned out to be the least trouble in the protocol. The input data does not impact the transformation as much as the other factors

b) The random matrices created during the protocol. Different runs of the transformation produce different qualities of results. There was, however, no detectable special structure to the matrices in the runs that produced worse results.

c) Number of peers. This is *definitely* a problem. If the number of players is higher than 8, the transformation almost always results in infeasible problems. Reasons for this are discussed in section 5.1.2.

d) Random numbers in general. Even limiting the random numbers to values as small as 100 produces infeasible problems after transformation for most test cases. Usable results are achieved limiting the random numbers to values as small as 20 or 10.

e) The LP solver. The numbers produced during the transformation can be so large that the solver cannot handle them correctly.

The perturbations that occur are mostly small and arguably acceptable for the application. A new evaluation might be necessary when considering applying the protocol to a different use case.

5.1.2. Scalability concerns

The concern that large random numbers might cause problems was observed to be a valid one. Problems occur already at a very small input size. When a problem occurs, this results in the linear program generated by the transformation being infeasible in most cases.

The problem is probably caused by very large numbers in the following way: As the matrices P and Q are constructed by multiplying the random matrices P_i and Q_i of every player, the values in these matrices increase quite quickly and dramatically depending on the number of participants. For Q this is quite easy to formally show. Consider $q_{\max}$ as the maximum random element possible of each Q_i. Then, as Q_i are all positive monomial matrices, Q can contain elements on the order of $q_{\max}^n$ where n is the number of parties.

Assume we set $q_{\max} = 1000 = 10^3$, Q can contain numbers as high as 10^{15} even if the number of players is limited to 5. This is, of course, an *upper* bound for the elements of Q. However, the matrix still gets multiplied by the constraint matrices which in turn might contain quite large numbers. Additionally, P also will get multiplied onto the result. And the P_i are not monomial matrices, which means that P's elements cannot be bounded by a $p_{\max}^n$, as the matrix product can always contain sums of element multiplications.

Numbers generated in the protocol can potentially get so large that a standard LP solver cannot handle them (they exceed double precision for floating point numbers).

5.1.3. Concrete results

Currently, however, another problem is that the multiplication protocol as performed by the underlying library also limits the number sizes effectively. This is because after multiplication of shares, reduction is necessary, as explained in section 2.4.4. This leads to additional multiplications (with scalars, internally) and additions which at a certain number size cause internal errors (mostly overflows into negative space inside the field) which make the results of the multiplication incorrect[1]. As was mentioned in section 3.2.2, the upper half of the field is reserved to express negative numbers in the field. When numbers get so large that they transit into the negative range, they are extracted incorrectly when converting them back to normal integer space.

The secret sharing library provides tests to evaluate the correctness and scalability of the communication protocol for multiplication. With those tests, we were able to determine the values for which multiplication starts producing incorrect results. The tests consist of 1000 runs for 50 sets of points (x_i) and measuring the number of sets for which there are incorrect results in any of the 1000 multiplications. The tests are further parametrized with the number of factors (repeated multiplications on the results of previous computations), the maximum value of those factors, and the number of players among the secret is shared.

An excerpt of the results is shown in table 5.1 and table 5.2. From table 5.2 it is clearly observable that for larger values or higher number of factors, the number of failures increases. The last row, for instance, shows that there was an error (after at most 1000 runs) multiplying 12 factors of maximum value 57 with 47 out of 50 sets of points. On the other hand, in table 5.1 it can be seen that the same number of factors with a smaller maximum

[1] From a theoretical perspective, there is no limitation on the size of numbers in multiplication, but the library internally uses *long*s to store field elements. This is partially due to the fact that the external solver will not accept larger numbers in any event.

value, causes no problems. The relationship between number of factors, maximum value and number of failures can be observed consistently. From appendix A it can be seen, that also the number of players has an influence here. With increasing player count more problems occur in multiplication.

To evaluate the transformation in its entirety, the test program described in section 4.2 was used. The goal was to test different test cases and different sets of parameters. These parameters were:

1. Size of the test cases (number of equality and inequality constraints, number of variables).
2. Maximum values (in the original input – M_1, M_2, x and c).
3. Maximum values (for the random matrices in the protocol).
4. *max_input* parameter of the library[2].
5. Number of players.

Ten test cases were generated for the following parameter sets:

a) 10 equality constraints; 10 inequality constraints; 10 variables; maximum value in the input problem: 500; maximum random values in the transformation: 20; *max_input* 60; 5 players;

b) 10 equality constraints; 10 inequality constraints; 10 variables; maximum value in the input problem: 500; maximum random values in the transformation: 10; *max_input* 30; 5 players;

c) 15 equality constraints; 10 inequality constraints; 15 variables; maximum value in the input problem: 500; maximum random values in the transformation: 10; *max_input* 30; 5 players;

d) 15 equality constraints; 10 inequality constraints; 30 variables; maximum value in the input problem: 200; maximum random values in the transformation: 10; *max_input* 30; 5 players;

e) 20 equality constraints; 10 inequality constraints; 20 variables; maximum value in the input problem: 200; maximum random values in the transformation: 10; *max_input* 30; 5 players;

f) 30 equality constraints; 10 inequality constraints; 30 variables; maximum value in the input problem: 200; maximum random values in the transformation: 10; *max_input* 30; 5 players;

g) 10 equality constraints; 10 inequality constraints; 10 variables; maximum value in the input problem: 500; maximum random values in the transformation: 10; *max_input* 30; 6 players;

[2]This parameter restricts the size of randomly-generated points (x_i) and all randomly-generated coefficients (recall that when sharing a secret S a random polynomial $p(x)$ with $p(0) = S$ is generated), including the coefficients of the $q_i(x)$ polynomials from the multiplication/reduction protocol.

# Players	Max. value	# Factors	Failed for · sets of points
5	50	2	0
5	50	3	0
5	50	4	0
5	50	5	0
5	50	6	0
5	50	7	0
5	50	8	0
5	50	9	0
5	50	10	0
5	50	11	0
5	50	12	3
5	51	2	0
5	51	3	0
5	51	4	0
5	51	5	0
5	51	6	0
5	51	7	0
5	51	8	0
5	51	9	0
5	51	10	0
5	51	11	0
5	51	12	9
5	52	2	0
5	52	3	0
5	52	4	0
5	52	5	0
5	52	6	0
5	52	7	0
5	52	8	0
5	52	9	0
5	52	10	0
5	52	11	0
5	52	12	16

Table 5.1.: A few results of the multiplication tests supplied by the library, showing fairly good results.

# Players	Max. value	# Factors	Failed for · sets of points
5	55	2	0
5	55	3	0
5	55	4	0
5	55	5	3
5	55	6	6
5	55	7	7
5	55	8	9
5	55	9	4
5	55	10	3
5	55	11	2
5	55	12	37
5	56	2	3
5	56	3	3
5	56	4	13
5	56	5	12
5	56	6	6
5	56	7	13
5	56	8	11
5	56	9	10
5	56	10	16
5	56	11	7
5	56	12	46
5	57	2	11
5	57	3	12
5	57	4	14
5	57	5	12
5	57	6	11
5	57	7	10
5	57	8	10
5	57	9	12
5	57	10	11
5	57	11	12
5	57	12	47

Table 5.2.: A few results of the multiplication tests supplied by the library, showing some more problematic cases.

h) 15 equality constraints; 10 inequality constraints; 15 variables; maximum value in the input problem: 500; maximum random values in the transformation: 10; *max_input* 30; 6 players;

i) 20 equality constraints; 10 inequality constraints; 20 variables; maximum value in the input problem: 500; maximum random values in the transformation: 10; *max_input* 30; 6 players;

In the following, let T be the set of test cases. For each $t \in T$, let ${}_tx$ be the vector of correct (optimal) solutions to the (untransformed) linear program t. let t'_j denote the transformed problem in run j. Let k be the number of variables. Let ${}_{(t,j)}x'$ denote the solution vector for the transformed test case t' of t in run j. Let F be the set of indices j for which t'_j is considered feasible by the solver.

For each set of parameters some statistical measurements were taken. We counted the number of runs which yielded results close to the real solution, differentiating between various levels of tolerance (0.001, 0.01, 0.05, 0.1, 0.5, 1.0 and 2.0). A result is said to be *in bounds* of a certain tolerance level ϵ, when all variables differ from the real result by at most ϵ, formally if it holds that $\forall_i \text{abs}({}_{(t,j)}x'_i - {}_tx_i) < \epsilon$.

Additionally, the number of transformed problems that were recognized as feasible by the solver ($|F|$) was recorded. For all feasible problems, the accumulated and average absolute error per variable was calculated. Formally,

$$E_{acc} = (e_1, \ldots, e_k), \text{ where } e_i = \sum_{j \in F} \text{abs}({}_{(t,j)}x'_i - {}_tx_i) \qquad \forall i \in \{1, \ldots, k\}$$

$$E_{avg} = (a_1, \ldots a_k), \text{ where } a_i = \frac{e_i}{|F|} \qquad \forall i \in \{1, \ldots, k\}.$$

An analogous error calculation was done for all the ${}_{(t,j)}x'$ that were in bounds of at least one tolerance level (2.0). This excludes the cases where some overflows in the underlying library have completely ruined the results. Let these accumulated and average absolute in-bounds-errors be denoted by $E_{B_{acc}}$ and $E_{B_{avg}}$, respectively. Let B_ϵ be the set of ${}_{(t,j)}x'$ that are in bounds with respect to tolerance level ϵ.

In table 5.3, the results for the parameter set a) are presented. We show the number of feasible t' ($|F|$) as well as how many of those are solved within bounds.

When examining these results it is quite obvious that the parameters were not very good. The number of feasible transformed problems is 0 in more than half of the cases.

In table 5.4, the results for the parameter set b) are presented in the same way as for a). In table 5.5, we show the average absolute in-bounds-error per variable $E_{B_{avg}}$ for those test cases.

As can be seen, the higher the tolerance, the more x' are considered in-bounds. And the probability of obtaining an acceptable solution using the transformation is remarkably high if we consider a solution with an error of (at most) 2 in every variable acceptable. This might be highly dependant on the application.

Comparing these results with those of parameter set a), it is clear that the maximum random value and the *max_input* parameter of the library have a great influence on the success probability of the algorithm. A value of 20 and 60, respectively, clearly were too large.

Testcase (t)	# runs	$\|F\|$	$\|B_{0.001}\|$	$\|B_{0.01}\|$	$\|B_{0.05}\|$	$\|B_{0.1}\|$	$\|B_{0.5}\|$	$\|B_{1.0}\|$	$\|B_{2.0}\|$
0	200	1	0	0	0	0	1	1	1
1	200	1	0	0	0	0	1	1	1
2	200	0	0	0	0	0	0	0	0
3	200	0	0	0	0	0	0	0	0
4	200	2	0	0	0	0	2	2	2
5	200	1	0	0	0	0	1	1	1
6	200	0	0	0	0	0	0	0	0
7	200	0	0	0	0	0	0	0	0
8	200	0	0	0	0	0	0	0	0
9	200	0	0	0	0	0	0	0	0

Table 5.3.: Results for parameter set a) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

Testcase (t)	# runs	$\|F\|$	$\|B_{0.001}\|$	$\|B_{0.01}\|$	$\|B_{0.05}\|$	$\|B_{0.1}\|$	$\|B_{0.5}\|$	$\|B_{1.0}\|$	$\|B_{2.0}\|$
0	1000	959	0	11	330	642	932	949	956
1	1000	981	0	18	448	813	974	979	981
2	1000	983	0	12	465	806	977	983	983
3	1000	992	0	11	180	405	807	904	956
4	1000	952	0	21	445	763	943	950	952
5	1000	982	0	14	372	703	970	980	982
6	1000	977	0	7	321	658	945	967	975
7	1000	992	0	18	501	840	989	992	992
8	1000	978	0	14	442	751	972	975	976
9	1000	960	0	9	344	607	906	939	952

Table 5.4.: Results for parameter set b) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

In table 5.6 and table 5.7, the results for the parameter set c) are presented in the same manner. The difference that can be directly observed in comparison to parameter set b) is that the number of "better" results is significantly smaller while the number of infeasible transformed problems seems to be smaller, too.

In table 5.8 and table 5.9, the results for the parameter set d) are presented in a similar way. Note that table 5.9 does not show the average absolute in-bounds-error per variable $E_{B_{avg}}$, but the average absolute error per variable E_{avg}, as too few results are actually in bounds.

The results here show that with the number of variables the error probability rises. This is logical, as more variables mean larger matrices and thus larger values both in and after the transformation, leading to the problems with multiplication discussed above. However, there is also the possibility that the difference between the number of equality constraints and the number of variables result in an LP which is not "bounded enough" after the transformation, causing unpredictable behaviour in the solver. This possibility seems likely, when comparing these results to those of parameter set f).

Testcase (t)	$E_{B_{avg}}$
0	(0.0287, 0.0303, 0.0243, 0.0235, 0.0267, 0.0748, 0.0288, 0.0369, 0.0323, 0.0684)
1	(0.0256, 0.0277, 0.0205, 0.0200, 0.0191, 0.0213, 0.0242, 0.0215, 0.0222, 0.0233)
2	(0.0159, 0.0235, 0.0208, 0.0214, 0.0288, 0.0189, 0.0181, 0.0213, 0.0175, 0.0297)
3	(0.1535, 0.0737, 0.0995, 0.0718, 0.0838, 0.1070, 0.2302, 0.1458, 0.0516, 0.1766)
4	(0.0256, 0.0202, 0.0305, 0.0243, 0.0274, 0.0247, 0.0266, 0.0247, 0.0153, 0.0263)
5	(0.0247, 0.0263, 0.0348, 0.0290, 0.0251, 0.0470, 0.0331, 0.0292, 0.0330, 0.0311)
6	(0.0348, 0.0351, 0.0465, 0.0434, 0.0434, 0.0362, 0.0592, 0.0456, 0.0514, 0.0390)
7	(0.0180, 0.0186, 0.0176, 0.0198, 0.0169, 0.0166, 0.0186, 0.0204, 0.0188, 0.0193)
8	(0.0225, 0.0265, 0.0266, 0.0241, 0.0254, 0.0239, 0.0250, 0.0232, 0.0232, 0.0198)
9	(0.0848, 0.0675, 0.0314, 0.0892, 0.0843, 0.0503, 0.0295, 0.0670, 0.0437, 0.0574)

Table 5.5.: Average absolute in-bounds-errors per variable (rounded to 4 decimal digits) for test cases with parameter set b).

Testcase (t)	# runs	$\lvert F\rvert$	$\lvert B_{0.001}\rvert$	$\lvert B_{0.01}\rvert$	$\lvert B_{0.05}\rvert$	$\lvert B_{0.1}\rvert$	$\lvert B_{0.5}\rvert$	$\lvert B_{1.0}\rvert$	$\lvert B_{2.0}\rvert$
0	1000	996	0	4	236	607	985	993	995
1	1000	999	0	0	82	362	890	961	990
2	1000	997	0	3	165	456	935	982	993
3	1000	998	0	0	191	528	975	992	998
4	1000	994	0	1	140	453	955	984	992
5	1000	998	0	1	181	522	965	993	996
6	1000	1000	0	5	306	700	992	998	999
7	1000	999	0	0	221	591	975	996	999
8	1000	1000	0	1	206	560	964	989	996
9	1000	1000	0	1	173	483	957	984	997

Table 5.6.: Results for parameter set c) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

In table 5.10 and table 5.11, the results for the parameter set e) are presented in the same manner as for b) and c). The results look fairly similar to those of parameter set c). This is probably related to the fact that in both cases the number of variables matches the number of equality constraints.

In table 5.12, the results for the parameter set f) are presented in the same manner as for b), c), and e). The results resemble those of parameter sets c) and e), again, probably a result of the similar relation between number of variables and number of equality constraints.

Comparing the results of parameter set d) to these here, it is clear that the latter constitute a significant improvement over the former. This is a strong indication that the relation between number of constraints and number of variables is significant to the correctness. The possibility should be considered that the test cases in parameter set d) were actually unbounded.

In table 5.13 and table 5.14, the results for the parameter set g) are presented. This parameter set differs from b) only in the number of players (and the number of runs on each test case, but that does not have an impact here), and it is clearly observable that this

Testcase (t)	$E_{B_{avg}}$
0	(0.0275, 0.0324, 0.0412, 0.0322, 0.0300, 0.0467, 0.0360, 0.0397, 0.0297, 0.0379, 0.0402, 0.0321, 0.0367, 0.0333, 0.0363)
1	(0.0935, 0.0793, 0.1112, 0.1607, 0.0556, 0.0813, 0.1081, 0.0685, 0.0806, 0.0923, 0.0977, 0.1061, 0.1058, 0.0649, 0.0785)
2	(0.0925, 0.0815, 0.0432, 0.0515, 0.0621, 0.0581, 0.0847, 0.0874, 0.0740, 0.0428, 0.0903, 0.0723, 0.0503, 0.0629, 0.0759)
3	(0.0364, 0.0744, 0.0406, 0.0412, 0.0379, 0.0571, 0.0361, 0.0321, 0.0335, 0.0678, 0.0368, 0.0603, 0.0429, 0.0375, 0.0419)
4	(0.0447, 0.0527, 0.0534, 0.0397, 0.0484, 0.0624, 0.0517, 0.0954, 0.0604, 0.0562, 0.0475, 0.0706, 0.0527, 0.0756, 0.0744)
5	(0.0678, 0.0519, 0.0489, 0.0468, 0.0635, 0.0405, 0.0426, 0.0477, 0.0400, 0.0537, 0.0499, 0.0397, 0.0454, 0.0554, 0.0545)
6	(0.0287, 0.0259, 0.0270, 0.0299, 0.0243, 0.0244, 0.0282, 0.0328, 0.0240, 0.0344, 0.0257, 0.0306, 0.0288, 0.0257, 0.0281)
7	(0.0692, 0.0369, 0.0351, 0.0341, 0.0367, 0.0353, 0.0339, 0.0406, 0.0583, 0.0350, 0.0315, 0.0301, 0.0344, 0.0266, 0.0351)
8	(0.0475, 0.0681, 0.0613, 0.0683, 0.0491, 0.0451, 0.0447, 0.0418, 0.0381, 0.0357, 0.0328, 0.0518, 0.0377, 0.0594, 0.0323)
9	(0.0443, 0.0800, 0.0692, 0.0642, 0.0577, 0.0610, 0.0682, 0.0394, 0.0559, 0.0582, 0.0741, 0.0433, 0.0366, 0.0538, 0.0434)

Table 5.7.: Average absolute in-bounds-errors per variable (rounded to 4 decimal digits) for test cases with parameter set c).

difference has a significant influence. While for b) approximately 98% of the transformed problems were considered feasible by the solver, in this parameter set it is only roughly 85%. This makes sense when recalling that the number of unproblematic multiplications decreases with increasing player number.

In table 5.15 and table 5.16, the results for the parameter set h) are presented. This set correspond to parameter set c) except that the number of players, previously 5, is 6 in these test cases. The same problem as with parameter set g) can be observed, only more drastic. The number of feasible transformed problems is even lower. This is most likely a result of overflows during multiplication and reduction in the library, as more multiplications occur for 6 players.

In table 5.17 and table 5.18, the results for the parameter set i) are presented. This parameter set was clearly too optimistic. Only 2 of the test cases are even transformed to a recognizably feasible problem, and each only in 1 of 200 runs of the transformation. The problems discussed before obviously have a strong impact here, as 6 players are executing the protocol and the maximum input number is 500 in a large linear program (with 20 equality and 10 inequality constraints, and 20 variables) that is passed to the transformation program.

Testcase (t)	# runs	$\|F\|$	$\|B_{0.001}\|$	$\|B_{0.01}\|$	$\|B_{0.05}\|$	$\|B_{0.1}\|$	$\|B_{0.5}\|$	$\|B_{1.0}\|$	$\|B_{2.0}\|$
0	200	200	0	0	0	0	1	1	2
1	200	200	0	0	0	1	1	1	2
2	200	200	0	0	0	1	1	1	1
3	200	200	0	0	0	0	0	0	0
4	200	200	0	0	0	1	1	1	1
5	200	199	0	0	0	0	0	0	0
6	200	200	0	0	1	1	1	1	1
7	200	200	0	0	0	0	0	0	0
8	200	200	0	0	0	0	0	0	0
9	200	199	0	0	0	0	0	0	0

Table 5.8.: Results for parameter set d) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

Testcase (t)	E_{avg}
0	(343.7227, 320.2237, 382.6472, 321.2892, 425.4275, 381.2259, 359.5327, 390.2501, 341.3129, 363.5446, 398.8473, 412.1795, 339.422, 408.0418, 373.0213, 319.2119, 322.0907, 267.549, 421.0882, 332.943, 345.9597, 353.064, 356.699, 326.0166, 406.4146, 301.8166, 365.7971, 352.583, 372.3261, 338.5368)
...	...(omitted, not relevant)

Table 5.9.: Average absolute errors per variable (rounded to 4 decimal digits) for test cases with parameter set d). Note these are of all feasible transformed LPs, not just those that yielded in-bounds results.

5.2. Discussion

From the results in section 5.1, it is clear that there are a number of parameters that influence the outcome of the algorithm. As was predicted, tests with the parameter set a) confirmed that larger random numbers cause trouble and make the transformed problems infeasible in most cases.

It was also observed that certain relations between number of constraints and number of variables have a large impact on the results. A high number of variables and a relatively low number of constraints resulted in almost no solution in bounds. Again, this makes sense if the feasible region is unbounded in some direction. The transformed problem might then also be unbounded and, because of small errors, the solutions might diverge from the solution to the input problem. The exact reason, however, remains unknown.

It needs to be emphasized that in cases where no obvious errors occurred that made the results unusable, the solutions obtained were nearly always quite accurate.

The results also show the amount of transformed problems that give an acceptable solution, where "acceptable" is left to be defined by the reader, as this is almost certainly application-specific. For the supply chain master planning use case, an error of 2 might arguably be acceptable when variable values are on the order of several thousands. How-

Testcase (t)	# runs	$\lvert F\rvert$	$\lvert B_{0.001}\rvert$	$\lvert B_{0.01}\rvert$	$\lvert B_{0.05}\rvert$	$\lvert B_{0.1}\rvert$	$\lvert B_{0.5}\rvert$	$\lvert B_{1.0}\rvert$	$\lvert B_{2.0}\rvert$
0	200	200	0	0	7	53	189	198	200
1	200	200	0	0	25	87	191	199	200
2	200	199	0	0	19	93	196	198	199
3	200	200	0	0	19	66	181	196	199
4	200	200	0	0	2	34	161	185	197
5	200	200	0	0	21	77	192	197	200
6	200	200	0	0	14	72	190	198	200
7	200	200	0	0	14	57	177	198	200
8	200	200	0	0	27	88	196	199	200
9	200	200	0	0	18	97	197	200	200

Table 5.10.: Results for parameter set e) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

ever, when the variable values are so large, success probability of the protocol has decreased. Exceptionally good results are rare with all sets of parameters.

Improving the algorithm's practical performance would require the solution to multiple problems encountered when generating tests. The issue with needing to bound random numbers to a fixed small size in order to get feasible results might be solved by using an "exponential" distribution in the random number generator to keep numbers relatively small without setting a hard limit at 10 or 20, as suggested by Christian Rupprecht [23].

Another possibility to avoid most of the problems discussed here with good chances for success would be to change the library to support arbitrary sized integers – possibly by using GMP, the The GNU Multiple Precision Arithmetic Library[3] (in order to set the prime field size high enough to avoid issues with field overflow on reasonably-sized inputs for the use case) and using a solver that can also deal with arbitrary precision. These measures will most certainly increase the runtime but have the potential to improve the viability of the algorithm. As many use cases are not time-critical, for a large number of applications, trading inaccurate, incorrect or infeasible results with a fast solver for accurate results with a slow solver is an acceptable tradeoff.

To eliminate small errors in the results, it might be possible to run the algorithm several times and compute mean values of the obtained results, although this approach is questionable from a security perspective, as the knowledge of several M'' will give an attacker more information on the random matrices.

There remains one major limitation of the algorithm, one specifically imposed by the use of multiplicative secret sharing, and that is the limitation on the threshold t, a problem that cannot be overcome by any of the above methods. However, this is a known issue with multiplicative secret-sharing schemes and not one we attempted or intended to approach in this work.

[3] `https://gmplib.org/`

5.3. Conclusion

In this work, a transformation-based approach to secure private linear programming was examined and evaluated. In the process, a number of unexpected problems were encountered which changed both the proposed approach for testing the algorithm and its evaluation. The unexpected discovery that no secret sharing library supports multiplicative secret sharing, or even adequate field sizes for our application, was a major stumbling block, made even more surprising because the technique is described in a paper originally published in 1988 by Ben-Or et al. [3]. This required that a library to support multiplicative secret sharing be developed in close cooperation with this work; while this was achieved, it was far from trivial, and its results and limitations greatly impacted the implementation of the algorithm to be tested. The multiplication protocol was a particular challenge to implement, both from the library developer's perspective as well as in the transformation algorithm that used it. This was completely unanticipated, as secret sharing is considered by most to be an easy-to-understand, easy-to-implement concept, and the complexity of the multiplication protocol is rarely mentioned in literature, with the exception of Asharov and Lindell's detailed proof, which gives more insight into the complexity of the task and was only published in 2011.

The limitations on the size of the randomly-generated Lagrange polynomial coefficients and points for secret sharing and number of repeated multiplications inherent in the protocol were also problematic, as it became difficult to determine whether the implementation was incorrect or the algorithm was creating incorrect results through prime overflow.

In light of Laud and Pankova's recent research [17, 18], it would be interesting to re-evaluate the security of the algorithm. It may be possible that improvements to increase the practicality of the protocol are not useful if the security of transformation-based approaches is flawed in itself.

For future research, however, it would be interesting to use arbitrary precision libraries for the secret sharing library, the transformation program, and the solver. The real scalability issue here is that the number of players directly influences the number of consecutive multiplications and the size of the numbers that are fed to the solver. More interesting would be to see if there is a possibility to circumvent the threshold limitation discussed in section 3.1.1, however unlikely. This would make real-world usage of such protocols much more likely if there were stronger privacy guarantees for individual inputs.

Finally, if these improvements were found to be viable, the next step would be use the algorithm in a practical application, possibly building on the implementation produced for this thesis.

Testcase (t)	$E_{B_{avg}}$
0	(0.0715, 0.0717, 0.0739, 0.115, 0.0562, 0.0492, 0.0622, 0.1104, 0.0548, 0.0516, 0.1067, 0.0602, 0.1154, 0.0755, 0.0581, 0.0591, 0.0462, 0.0471, 0.0457, 0.0485)
1	(0.04, 0.0819, 0.0544, 0.0647, 0.0615, 0.0579, 0.0664, 0.0327, 0.0554, 0.0833, 0.0378, 0.0477, 0.044, 0.0456, 0.0674, 0.0513, 0.0643, 0.0523, 0.0403, 0.0381)
2	(0.06, 0.0547, 0.0399, 0.0551, 0.0417, 0.0675, 0.0389, 0.0552, 0.0574, 0.0461, 0.0654, 0.0444, 0.0426, 0.0419, 0.0555, 0.054, 0.0511, 0.0515, 0.0602, 0.0544)
3	(0.0724, 0.1027, 0.0794, 0.079, 0.0435, 0.0702, 0.0752, 0.0745, 0.0965, 0.0933, 0.0803, 0.0603, 0.0692, 0.0537, 0.062, 0.0611, 0.0581, 0.1037, 0.0652, 0.0808)
4	(0.0759, 0.1165, 0.0839, 0.081, 0.0827, 0.1931, 0.0906, 0.1278, 0.2071, 0.1053, 0.2054, 0.073, 0.1523, 0.0634, 0.1299, 0.0754, 0.0835, 0.1108, 0.0768, 0.0878)
5	(0.0466, 0.0399, 0.0411, 0.0595, 0.0753, 0.0481, 0.0475, 0.0564, 0.0416, 0.046, 0.0352, 0.0437, 0.1022, 0.0641, 0.0876, 0.0501, 0.0534, 0.0527, 0.0612, 0.0483)
6	(0.076, 0.118, 0.0355, 0.0465, 0.0299, 0.0659, 0.0607, 0.0491, 0.0575, 0.0506, 0.0569, 0.0619, 0.0443, 0.043, 0.0977, 0.1105, 0.0695, 0.0791, 0.0578, 0.0792)
7	(0.0487, 0.0504, 0.0644, 0.0627, 0.0809, 0.0667, 0.073, 0.0659, 0.1101, 0.1204, 0.0704, 0.1599, 0.1588, 0.0548, 0.0482, 0.1361, 0.1064, 0.0488, 0.0561, 0.1462)
8	(0.0337, 0.0669, 0.048, 0.0435, 0.0397, 0.0506, 0.0414, 0.0435, 0.0653, 0.0498, 0.0409, 0.0369, 0.0465, 0.03, 0.0487, 0.0353, 0.0366, 0.0408, 0.036, 0.0529)
9	(0.0349, 0.0412, 0.0424, 0.0627, 0.0454, 0.0542, 0.0333, 0.0422, 0.0494, 0.0488, 0.0616, 0.0462, 0.0341, 0.0377, 0.0384, 0.0309, 0.035, 0.0493, 0.0293, 0.0374)

Table 5.11.: Average absolute in-bounds-errors per variable (rounded to 4 decimal digits) for test cases with parameter set e).

Testcase (t)	# runs	$\|F\|$	$\|B_{0.001}\|$	$\|B_{0.01}\|$	$\|B_{0.05}\|$	$\|B_{0.1}\|$	$\|B_{0.5}\|$	$\|B_{1.0}\|$	$\|B_{2.0}\|$
0	200	200	0	0	1	19	167	190	199
1	200	200	0	0	3	32	183	200	200
2	200	200	0	0	5	39	189	198	200
3	200	200	0	0	3	21	165	191	199
4	200	200	0	0	0	3	119	177	196
5	200	200	0	0	2	18	159	193	197
6	200	200	0	0	3	53	189	198	200
7	200	200	0	0	9	45	188	198	200
8	200	200	0	0	6	59	195	200	200
9	200	200	0	0	5	41	191	198	200

Table 5.12.: Results for parameter set f) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

Testcase (t)	# runs	$\|F\|$	$\|B_{0.001}\|$	$\|B_{0.01}\|$	$\|B_{0.05}\|$	$\|B_{0.1}\|$	$\|B_{0.5}\|$	$\|B_{1.0}\|$	$\|B_{2.0}\|$
0	200	169	0	0	3	21	142	166	169
1	200	172	0	0	2	10	133	158	168
2	200	168	0	0	2	9	134	161	167
3	200	160	0	0	2	26	134	155	160
4	200	164	0	0	5	25	136	161	164
5	200	169	0	0	7	25	143	166	169
6	200	159	0	0	3	24	140	156	158
7	200	165	0	0	10	26	145	160	164
8	200	176	0	0	8	27	146	174	176
9	200	169	0	0	12	35	157	167	169

Table 5.13.: Results for parameter set g) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

Testcase (t)	$E_{B_{avg}}$
0	(0.0818, 0.0768, 0.097, 0.0784, 0.0715, 0.069, 0.128, 0.1058, 0.1173, 0.0725)
1	(0.0848, 0.0693, 0.1018, 0.143, 0.1776, 0.0807, 0.12, 0.1109, 0.178, 0.0944)
2	(0.0754, 0.0963, 0.0673, 0.0872, 0.1246, 0.076, 0.1106, 0.067, 0.0823, 0.0734)
3	(0.0719, 0.0773, 0.0676, 0.0862, 0.08, 0.117, 0.0871, 0.0835, 0.0877, 0.0707)
4	(0.0703, 0.0834, 0.0717, 0.0846, 0.0799, 0.0813, 0.0468, 0.0732, 0.0666, 0.0716)
5	(0.0662, 0.0532, 0.0777, 0.0593, 0.0608, 0.0667, 0.0713, 0.0794, 0.0789, 0.058)
6	(0.0672, 0.0531, 0.0542, 0.0575, 0.0671, 0.069, 0.0616, 0.0741, 0.061, 0.0633)
7	(0.0762, 0.0787, 0.1006, 0.0512, 0.0626, 0.0726, 0.0605, 0.0598, 0.0696, 0.0591)
8	(0.0745, 0.0575, 0.0632, 0.0798, 0.0585, 0.0743, 0.0762, 0.0686, 0.0831, 0.0604)
9	(0.075, 0.0555, 0.0644, 0.0573, 0.0549, 0.0554, 0.0399, 0.0618, 0.051, 0.0618)

Table 5.14.: Average absolute in-bounds-errors per variable (rounded to 4 decimal digits) for test cases with parameter set g).

Testcase (t)	# runs	$\|F\|$	$\|B_{0.001}\|$	$\|B_{0.01}\|$	$\|B_{0.05}\|$	$\|B_{0.1}\|$	$\|B_{0.5}\|$	$\|B_{1.0}\|$	$\|B_{2.0}\|$
0	200	25	0	0	0	4	22	23	25
1	200	32	0	0	0	2	23	29	32
2	200	28	0	0	0	2	26	27	28
3	200	21	0	0	0	1	19	21	21
4	200	30	0	0	1	7	24	30	30
5	200	29	0	0	0	5	27	29	29
6	200	32	0	0	0	4	32	32	32
7	200	25	0	0	0	5	24	25	25
8	200	28	0	0	1	3	27	28	28
9	200	28	0	0	1	2	23	26	28

Table 5.15.: Results for parameter set h) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

Testcase (t)	$E_{B_{avg}}$
0	(0.1313, 0.0578, 0.0807, 0.1133, 0.2014, 0.0714, 0.0983, 0.1308, 0.1028, 0.0584, 0.1099, 0.0876, 0.0662, 0.0568, 0.0714)
1	(0.0643, 0.1249, 0.0631, 0.0907, 0.2931, 0.0688, 0.101, 0.0548, 0.2365, 0.0878, 0.1748, 0.0748, 0.2616, 0.0797, 0.0783)
2	(0.0522, 0.0456, 0.0534, 0.0895, 0.0302, 0.0658, 0.0575, 0.0686, 0.0359, 0.0283, 0.0477, 0.0336, 0.0549, 0.0316, 0.0514)
3	(0.1172, 0.0772, 0.117, 0.1073, 0.1411, 0.0439, 0.1029, 0.0561, 0.0424, 0.1175, 0.0679, 0.0735, 0.123, 0.1136, 0.1088)
4	(0.0422, 0.0795, 0.0866, 0.0557, 0.0781, 0.0772, 0.0554, 0.0803, 0.0751, 0.0339, 0.0615, 0.0735, 0.0386, 0.0523, 0.0503)
5	(0.0478, 0.0629, 0.0486, 0.0394, 0.0306, 0.0387, 0.0495, 0.0571, 0.0685, 0.0444, 0.0349, 0.0406, 0.0401, 0.058, 0.0396)
6	(0.046, 0.073, 0.039, 0.0539, 0.0324, 0.0586, 0.0385, 0.0532, 0.045, 0.0437, 0.0581, 0.0638, 0.0358, 0.0456, 0.0469)
7	(0.0693, 0.0469, 0.0931, 0.0707, 0.0554, 0.0491, 0.0539, 0.0655, 0.0517, 0.1461, 0.0491, 0.0601, 0.0292, 0.06, 0.0364)
8	(0.0608, 0.0895, 0.0405, 0.0934, 0.0578, 0.0449, 0.0478, 0.0798, 0.0616, 0.0481, 0.0623, 0.0307, 0.0369, 0.0686, 0.0594)
9	(0.1465, 0.1091, 0.12, 0.046, 0.1437, 0.0486, 0.2078, 0.1178, 0.0574, 0.0851, 0.0633, 0.1175, 0.0769, 0.1063, 0.0891)

Table 5.16.: Average absolute in-bounds-errors per variable (rounded to 4 decimal digits) for test cases with parameter set h).

Testcase (t)	# runs	$\|F\|$	$\|B_{0.001}\|$	$\|B_{0.01}\|$	$\|B_{0.05}\|$	$\|B_{0.1}\|$	$\|B_{0.5}\|$	$\|B_{1.0}\|$	$\|B_{2.0}\|$
0	200	0	0	0	0	0	0	0	0
1	200	1	0	0	0	0	1	1	1
2	200	0	0	0	0	0	0	0	0
3	200	0	0	0	0	0	0	0	0
4	200	0	0	0	0	0	0	0	0
5	200	0	0	0	0	0	0	0	0
6	200	0	0	0	0	0	0	0	0
7	200	1	0	0	0	0	1	1	1
8	200	0	0	0	0	0	0	0	0
9	200	0	0	0	0	0	0	0	0

Table 5.17.: Results for parameter set i) showing the number of feasible transformed LPs and the numbers of in-bounds solutions for different tolerance levels.

Testcase (t)	$E_{B_{avg}}$
0	(cannot be calculated – no solution was in bounds or even feasible)
1	(0.031, 0.0208, 0.0162, 0.02, 0.013, 0.012, 0.005, 0.002, 0.103, 0.0246, 0.0475, 0.016, 0.0032, 0.024, 0.0093, 0.0013, 0.0, 0.04, 0.0279, 0.017)
2	(as with $t = 0$)
3	(as with $t = 0$)
4	(as with $t = 0$)
5	(as with $t = 0$)
6	(as with $t = 0$)
7	(0.0016, 0.1, 0.1459, 0.0048, 0.0938, 0.0749, 0.0642, 0.12, 0.038, 0.058, 0.086, 0.0119, 0.0256, 0.0528, 0.1392, 0.0304, 0.0832, 0.0288, 0.0256, 0.057)
8	(as with $t = 0$)
9	(as with $t = 0$)

Table 5.18.: Average absolute in-bounds-errors per variable (rounded to 4 decimal digits) for test cases with parameter set i).

Appendix

A. Detailed test results for multiplication

These are some more results from the multiplication tests provided by the library.

# Players	Max. value	# Factors	Failed for · sets of points
7	12	2	0
7	12	3	0
7	12	4	0
7	12	5	0
7	12	6	0
7	12	7	0
7	12	8	0
7	12	9	0
7	12	10	0
7	12	11	0
7	12	12	0
7	13	2	0
7	13	3	0
7	13	4	0
7	13	5	0
7	13	6	0
7	13	7	0
7	13	8	0
7	13	9	0
7	13	10	0
7	13	11	0
7	13	12	0
7	14	2	0
7	14	3	0
7	14	4	0
7	14	5	0
7	14	6	0
7	14	7	0
7	14	8	0
7	14	9	0
7	14	10	0
7	14	11	0
7	14	12	0
7	15	2	0
7	15	3	0

# Players	Max. value	# Factors	Failed for · sets of points
7	15	4	0
7	15	5	0
7	15	6	0
7	15	7	0
7	15	8	0
7	15	9	0
7	15	10	0
7	15	11	0
7	15	12	0
7	16	2	0
7	16	3	0
7	16	4	0
7	16	5	0
7	16	6	0
7	16	7	0
7	16	8	0
7	16	9	0
7	16	10	0
7	16	11	0
7	16	12	0
7	17	2	0
7	17	3	0
7	17	4	0
7	17	5	0
7	17	6	0
7	17	7	0
7	17	8	0
7	17	9	0
7	17	10	0
7	17	11	0
7	17	12	0
7	18	2	22
7	18	3	17
7	18	4	22
7	18	5	24
7	18	6	17
7	18	7	21
7	18	8	19
7	18	9	19
7	18	10	25
7	18	11	23
7	18	12	17
7	19	2	30

# Players	Max. value	# Factors	Failed for · sets of points
7	19	3	34
7	19	4	33
7	19	5	38
7	19	6	29
7	19	7	33
7	19	8	32
7	19	9	29
7	19	10	31
7	19	11	32
7	19	12	33
7	20	2	37
7	20	3	36
7	20	4	39
7	20	5	33
7	20	6	37
7	20	7	38
7	20	8	36
7	20	9	39
7	20	10	37
7	20	11	37
7	20	12	47
7	21	2	41
7	21	3	44
7	21	4	44
7	21	5	48
7	21	6	44
7	21	7	46
7	21	8	48
7	21	9	42
7	21	10	47
7	21	11	46
7	21	12	40
7	22	2	47
7	22	3	47
7	22	4	48
7	22	5	48
7	22	6	45
7	22	7	46
7	22	8	44
7	22	9	45
7	22	10	50
7	22	11	49
7	22	12	48

# Players	Max. value	# Factors	Failed for · sets of points
7	23	2	48
7	23	3	48
7	23	4	49
7	23	5	48
7	23	6	47
7	23	7	47
7	23	8	49
7	23	9	47
7	23	10	50
7	23	11	49
7	23	12	48
7	24	2	50
7	24	3	49
7	24	4	48
7	24	5	46
7	24	6	48
7	24	7	46
7	24	8	45
7	24	9	50
7	24	10	49
7	24	11	50
7	24	12	49
7	25	2	50
7	25	3	50
7	25	4	50
7	25	5	49
7	25	6	49
7	25	7	48
7	25	8	49
7	25	9	50
7	25	10	48
7	25	11	49
7	25	12	49

Bibliography

[1] Gilad Asharov and Yehuda Lindell. A full proof of the bgw protocol for perfectly-secure multiparty computation, 2011.

[2] Alice Bednarz, Nigel Bean, and Matthew Roughan. Hiccups on the road to privacy-preserving linear programming. In *Proceedings of the 8th ACM workshop on Privacy in the electronic society*, WPES '09, pages 117–120, New York, NY, USA, 2009. ACM.

[3] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, STOC '88, pages 1–10, New York, NY, USA, 1988. ACM.

[4] Dennis Bennett. personal communication.

[5] M. Blum. Three applications of the oblivious transfer. 1981.

[6] Hans-Joachim Bungartz. 1. motivation and introduction: Numerical algorithms in computer science. Lecture slides (Numerisches Programmieren), 2012.

[7] Octavian Catrina and Sebastiaan De Hoogh. Secure multiparty linear programming using fixed-point arithmetic. In *Proceedings of the 15th European conference on Research in computer security*, ESORICS'10, pages 134–150, Berlin, Heidelberg, 2010. Springer-Verlag.

[8] Cowles Commission for Research in Economics. *Activity analysis of production and allocation*. John Wiley and Sons, New York, 1951.

[9] George Dantzig and A. Orden. A duality theorem based on the simplex method. In *Symposium on Linear Inequalities, USAF-Hq., SCOOP Publication No. 10*, pages pages 51–55, April 1952.

[10] Jannik Dreier and Florian Kerschbaum. Practical privacy-preserving multiparty linear programming based on problem transformation. In *Privacy, security, risk and trust (passat), 2011 ieee third international conference on and 2011 ieee third international conference on social computing (socialcom)*, pages 916–924, 2011.

[11] Wenliang Du. *A study of several specific secure two-party computation problems*. PhD thesis, West Lafayette, IN, USA, 2001. AAI3043719.

[12] Christian Grothoff. personal communication.

[13] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, second edition, 2002.

[14] William Kahan, Joseph D Darcy, Elect Eng, and High-Performance Network Computing. How java's floating-point hurts everyone everywhere.

[15] E.D. Karnin, J. Greene, and M.E. Hellman. On secret sharing systems. *Information Theory, IEEE Transactions on*, 29(1):35–41, 1983.

[16] Gerald Knizia, Wenbin Li, Sven Simon, and Hans-Joachim Werner. Determining the numerical stability of quantum chemistry algorithms. *Journal of Chemical Theory and Computation*, 7(8):2387–2398, 2011.

[17] Peeter Laud and Alisa Pankova. New attacks against transformation-based privacy-preserving linear programming. *IACR Cryptology ePrint Archive*, 2013:355, 2013.

[18] Peeter Laud and Alisa Pankova. On the (im)possibility of privately outsourcing linear programming. In *Proceedings of the 2013 ACM Workshop on Cloud Computing Security Workshop*, CCSW '13, pages 55–64, New York, NY, USA, 2013. ACM.

[19] Jiangtao Li and Mikhail J. Atallah. Secure and private collaborative linear programming. In *Collaborative Computing: Networking, Applications and Worksharing, 2006. CollaborateCom 2006. International Conference on*, pages 1–8, 2006.

[20] Yehuda Lindell and Benny Pinkas. Secure multiparty computation for privacy-preserving data mining. Cryptology ePrint Archive, Report 2008/197, 2008. `http://eprint.iacr.org/`.

[21] David G. Luenberger. *Linear and Nonlinear Programming*. New York: Springer, 3rd edition, 2008.

[22] Thomas Harvey Rowan. *Functional Stability Analysis of Numerical Algorithms*. PhD thesis, University of Texas, Austin, 1990.

[23] Christian Rupprecht. Personal communication, December 2013.

[24] Bruce Schneier. *Applied cryptography (2nd ed.): protocols, algorithms, and source code in C*. John Wiley & Sons, Inc., New York, NY, USA, 1995.

[25] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979.

[26] Adi Shamir, Ronald L. Rivest, and Leonard M. Adleman. Mental poker. Technical Report MIT-LCS-TM-125,Massachusetts Institute of Technology, 1979.

[27] Hartmut Stadtler. Supply chain management and advanced planning–basics, overview and challenges. *European Journal of Operational Research*, 163(3):575–588, June 2005.

[28] Tomas Toft. Solving linear programs using multiparty computation. In Roger Dingledine and Philippe Golle, editors, *Financial Cryptography and Data Security*, volume 5628 of *Lecture Notes in Computer Science*, pages 90–107. Springer Berlin Heidelberg, 2009.

[29] Jaideep Vaidya. Privacy-preserving linear programming. In *Proceedings of the 2009 ACM symposium on Applied Computing*, SAC '09, pages 2002–2007, New York, NY, USA, 2009. ACM.

[30] Roland Wunderling. *Paralleler und objektorientierter Simplex-Algorithmus*. PhD thesis, Technische Universität Berlin, 1996. `http://www.zib.de/Publications/abstracts/TR-96-09/`.

[31] Andrew C. Yao. Protocols for secure computations. In *IEEE Symposium on Foundations of Computer Science*, pages 160–164, 1982.

[32] Hang Zhou. A generic automatic numerical stability testing method. Master's thesis, 2010.